Thammasat University

Measuring AI Performance and Selecting Models Under Quality and Cost Constraints

Framework Proposal and Empirical Case Studies

Pacharapol Pimpa

Software Engineering · Bachelor's Degree Program

Independent Research Manuscript — Not Peer Reviewed

11 October 2026

Research version 1.0.0

Abstract

Selecting an AI configuration requires task-specific quality evidence and explicit operational limits. This paper proposes a unified task-conditioned framework that minimizes projected total cost among configurations with sufficient evidence of acceptable quality, bounded degradation relative to a reference, and compliance with budget and resource constraints. Four components—quality measurement, cost estimation, admissibility and optimization—are expressed through one central constrained optimization problem. The framework combines established decision principles; it neither measures universal intelligence nor establishes a new optimization theory. A completed Azure study of weighted pipeline selection on Dry Bean classification and sequential Bike Sharing forecasting provides supporting evidence about reproducibility and selection trade-offs, rather than direct validation of the proposed optimizer. The weighted rule selected linear models for Bean and boosting for Bike. Its Bike underprediction-heavy profile had higher held-out simulated loss than linear: difference 7.5731 weighted-count units/hour, adjusted conditional interval [3.0698, 12.1578]. Other comparisons were identical, inconclusive or failed a precision target. A separate, prospective Python code-generation protocol specifies independent development, selection and held-out sets, supported thinking configurations, paired quality assessment and complete cost accounting. Whether the proposed framework achieves lower cost while preserving acceptable quality remains an empirical question.

Keywords: task-conditioned evaluation; constrained model selection; non-inferiority; inference configuration; operational cost; reproducibility.

1. Introduction

An AI model is useful only relative to a task, its target population and its operating conditions. A quality score alone does not establish that a service meets latency, memory, quota or spending limits. Conversely, a cheaper configuration is unsuitable when its predictive quality falls below the task's required level. Choosing a model and an inference policy therefore requires a decision rule that states both the quality that must be preserved and the cost that should be minimized.

The central research question is: Can a task-conditioned optimization framework select lower-cost AI models and inference configurations while preserving acceptable task performance under operational constraints? The proposed answer is a testable decision procedure, not a claim that cost reductions have already been demonstrated. A complete configuration includes the provider or execution backend, model and immutable version, supported inference effort, preprocessing or prompt, decoding, resource allocation and execution policy. Retries, routing, verification and escalation change both quality and cost and belong to that configuration.

The original study implemented a hard-gated weighted selector for small predictive pipelines. Its completed experiments remain useful because they expose preference sensitivity, temporal degradation, negative outcomes and reproducibility requirements. They did not evaluate LLM inference effort, confidence-bound admission or complete monetary savings. The paper separates that empirical study from the proposed extension through a single admissibility-constrained cost objective.

"Unified" refers to a common evidence and decision interface. Metrics retain their task meanings: a macro-F1 value for bean classification is not comparable to a code-correctness proportion or a rental-count error. Likewise, the framework's minimum is conditional on a finite evaluated candidate set, a workload horizon and declared assumptions. It does not identify the universally best AI system.

2. Research Questions and Contributions

RQ1 asks whether a common evidence contract can represent task quality, operational costs and mandatory constraints without concealing incompatibility or missing evidence. RQ2 asks whether the proposed least-cost admissible selector reduces complete operational cost relative to a fixed reference while meeting a prespecified absolute quality floor and non-inferiority margin on untouched cases. RQ3 asks how decisions

and their uncertainty change with quality thresholds, budgets, workload composition and supported inference policies.

The contribution is an explicit specification linking task measurement, full-horizon cost, uncertainty-aware admission and deterministic selection. It includes a decision record that can support a reusable developer package. A second contribution is the preserved two-task case study, which demonstrates implementation traceability and documents adverse and inconclusive weighted-selection outcomes. A third is a prospective code-generation validation protocol designed to test RQ2 rather than infer it from unrelated experiments. The interface design and integrated specification are practical contributions; superiority over existing selection or routing approaches remains unestablished.

RQ1 is partially addressed by historical task adapters and evidence checks; a versioned implementation of the full quality/cost contract, module tests and externally inspectable decision receipts are still needed. RQ2 is unanswered: it requires frozen model/effort selection, untouched paired quality confirmation and complete cost comparison against the reference. RQ3 is partly addressed only for historical weighted-profile preferences and resource limits; quality-bound, budget, workload and inference-mode sensitivity require the new study. No implemented or validated model-and-effort package is claimed.

3. Related Work

Cost-sensitive learning connects predictive information to the consequences of decisions [1,2]. Constrained multi-objective AutoML and preference-based utilities already formalize resource–quality trade-offs [3,4]. These foundations explain why mandatory constraints should precede a compensatory ranking. They also limit novelty: writing a constrained argmin or an additive utility does not establish a new mathematical theory.

FrugalGPT studies cost-aware use of language models, including cascades [12]. RouteLLM studies learned routing between stronger and weaker models [13]. Thus low-cost model selection and quality–cost trade-offs are established research directions. This proposal emphasizes a finite configuration registry, absolute and reference-relative quality gates, an explicit insufficient-evidence state, full-horizon accounting and reproducible rejection reasons. It has not shown that these features outperform routing or cascading systems. A cascade or router can be one complete candidate policy and must be evaluated end to end.

Research on test-time computation shows that allocating inference computation can depend on problem difficulty [14]. This motivates evaluating supported inference modes rather than assuming that more effort always improves quality. Qwen3 provides documented thinking and non-thinking modes [15,16]. The present work does not infer a general effort–quality relationship from these capabilities.

The Benchmark Lottery [5], selection-bias analysis [6] and leakage research [7] motivate fixed task definitions and untouched assessment. Time-series bootstrap methods [8] motivate dependence-aware intervals but do not certify coverage for the rental series. Non-inferiority requires a prespecified margin and directional evidence; equivalence testing is a distinct established method [20]. Exact binomial confidence limits [21] provide a conservative building block for the proposed code-task admission test. MBPP [17,18] and stronger code-testing research [19] motivate a separate study; public benchmark splits cannot establish absence from model pretraining.

4. Problem Formulation

A task t defines inputs z, reference outcomes y, allowed outputs, a target population and a task-specific metric M_t. The deployment scenario fixes a horizon of N requests, a workload distribution, concurrency, hardware or API region, a currency, mandatory limits and a reference configuration c_{ref}. Difficulty groups are optional; they must be defined using input information available before prediction and frozen before final evaluation. Per-group decisions require adequate group-specific evidence and cannot borrow a pooled guarantee without justification.

A configuration c is immutable for the decision being evaluated. Its identifier binds model revision, inference mode, quantization, prompt or features, decoding, output-token limits, timeout, retry and fallback rules, tools, backend and resource allocation. Unsupported model–effort combinations are excluded rather than assigned hypothetical behavior. Policy failures, truncated outputs and exhausted retries remain part of its measured outcomes.

The raw registry records compatible, incompatible and insufficient-evidence candidates separately. In the master problem, $\mathcal{C}_t$ denotes the finite task-compatible set with complete, comparable evidence for the specified scenario. Completeness requires prediction coverage, metric definitions, paired reference evidence, price and unit provenance, bound methods and resource measurements. An empty evidence-ready set is a diagnostic result, not an invitation to impute missing values.

All quantities in one selection refer to the same scenario. Workload changes, model updates or tariffs invalidate relevant assumptions and require a new decision record. A model version number alone is insufficient when a provider can change service behavior.

5. Unified Mathematical Framework

5.1 Component 1: task-specific quality measurement

$$Q_t^D(c) \;=\; M_t\big((y_i,\, f_c(z_i))_{i \in D}\big) \qquad (1)$$

In Equation (1), t identifies the task, c the complete configuration, D a collection of distinct case identifiers and i one identifier. z_i is the input and y_i its reference outcome. The parenthesized indexed sequence preserves every case, including repeated input or outcome values; it does not deduplicate observations. $f_c(z_i)$ denotes the recorded output of the frozen policy for that run. For stochastic generation, retain problem and run identifiers and random seeds; repeated runs are nested within their problem, and paired candidates use the same problem identities. M_t computes the declared metric from this indexed record. Inputs and outcomes have task-specific units; indices are dimensionless. Q_t^D(c) is an observed estimate; Q_t(c) is the unknown population functional under the specified workload and generation distribution. A confidence statement additionally requires a defensible sampling model and treatment of dependence.

Use a higher-is-better orientation throughout the constraints. For classification, appropriate choices include accuracy, balanced accuracy, macro-F1 or a declared cost-sensitive measure; class imbalance and error consequences determine the choice. Macro-F1 averages class-level F1 values and is not a mean of per-case F1 scores; define the zero-denominator convention and resample cases or groups before recomputing the metric. For regression, negative MAE or negative RMSE preserves this orientation while retaining the original physical units. An MAE ceiling is represented by negating both the error metric and its threshold; quality differences then measure additional error in the original units.

For code generation, use the fraction of independently generated single completions passing a frozen test suite, with compilation errors, truncations and timeouts counted as failures. Tests are an imperfect correctness oracle. Repeated completions must have a specified sampling design and remain clustered by problem. Retrieval, summarization or other tasks require a validated metric or rubric and, where relevant, blinded ratings and rater agreement. Multiple mandatory quality criteria can be added as separate gates; a convenient composite score must not conceal a safety-critical failure. No normalization makes unrelated task metrics objectively comparable.

5.2 Component 2: operational cost estimation

$$C_N(c) = C_{setup}(c) + N \sum_h p_h E[v_h(c)] + C_{support,N}(c) \quad (2)$$

Equation (2) defines projected expected total cost for N requests in one currency. C_{setup}(c) allocates setup, loading or training expenses to that horizon. Category h has unit price p_h in currency per billable unit and consumption v_h(c) in billable units per request, including all attempts in the policy. E denotes expectation over the declared workload and execution randomness; usage measurements estimate this expectation. $C_{support,N}$(c) includes supporting charges over the same horizon. Each product p_h E[v_h(c)] has currency per request units, so multiplication by the request count N and addition of setup and support are dimensionally consistent. Categories and setup/support allocations must be mutually exclusive.

Non-overlapping categories include cached and uncached input tokens, billed generated tokens, rented GPU/CPU time, tools and transfer where applicable. Partition reasoning and final-output tokens according to the tariff: do not charge reasoning separately if it is already included in billed output. Count paid failures, retries, verification, routing and fallback. Allocate storage, networking, idle capacity, minimum charges, reservations and subscriptions explicitly. Model loading belongs either in setup or in a request/instance allocation, never both. For self-hosted inference, token counts measure usage; compute rental is the monetary charge, without an added hypothetical API tariff. Shared-infrastructure allocation must state whether costs are incremental or fully allocated. Constant p_h and N-scaled usage assume a fixed tariff and representative per-request workload; tiers, changing mix and time-varying prices require piecewise or request-specific accounting. Independence of requests is not required for linearity of expected cost, but is relevant to uncertainty estimation.

C_N(c) is an expected-cost projection estimated from usage, not an invoice. C_N^U(c) must identify what it bounds: an uncertain expected cost, a realized-horizon predictive allowance, or a deterministic capped expenditure. These meanings are not interchangeable. Record the bound method, assumptions and coverage separately from the quality family; a contingency percentage has no automatic statistical guarantee. Runtime spending stops complement a planning budget and belong to the evaluated policy. Reconcile actual expenditure from provider billing, discounts and delayed supporting-service charges. The historical warm CPU-dollar proxy covers only a small part of Equation (2).

5.3 Components 3 and 4: admissibility and least-cost optimization

The decision minimizes projected expected cost after both quality-preservation and operational-admissibility requirements are satisfied:

$$c^* = arg\,min_{c \in \mathcal{C}_t} C_N(c)$$
$$s.t. \quad L_\alpha[Q_t(c)] \geq q_{min}, \quad U_\alpha[Q_t(c_{ref}) - Q_t(c)] \leq \delta$$
$$C_N^U(c) \leq B_N, \quad r_l^U(c) \leq b_l, \quad \forall\, l \quad (3)$$

In Equation (3), c* is the final selected configuration and $\mathcal{C}_t$ is the finite task-compatible, evidence-ready set defined in Section 4. Strictly, argmin defines a set of minimizers; the deterministic tie-break below returns one member. Optimality is restricted to this set and workload. L_α and U_α are one-sided quality bounds constructed under a declared family-level error allowance alpha. Allocate claim-level error across candidates and required quality statements when simultaneous coverage is asserted; reusing alpha unadjusted for each claim does not control that family. The absolute floor q_{min} and reference-relative margin delta >= 0 use the task metric units

and are fixed before selection. For proportions, delta = 0.05 means five percentage points, not a five-percent relative change. Cost/resource bounds require their own stated deterministic or probabilistic interpretation; the quality-family allowance alone does not establish joint coverage of all operational constraints.

The reference-minus-candidate constraint tests non-inferiority: its upper bound must be at most the prespecified degradation margin delta. Pair outcomes on common cases; the reference is an anchor, not automatically the best or admissible configuration. Non-significance against zero is not evidence of non-inferiority or equivalence [20]. An interval crossing delta leaves admission inconclusive. Setting delta = 0 asks for evidence of no population-quality loss under the stated model, not identical responses. Reference-versus-itself degradation is exactly zero by identity; this does not establish equivalence between different policies.

C_N^U is a conservative upper cost bound, with superscript U denoting a bound rather than a power. B_N is the maximum operational budget in the same currency and horizon. Index l identifies a resource constraint; the final quantifier means every declared constraint. Resource l has a positive limit b_l and conservative bound r_l^U in matching units, such as milliseconds for request p95 latency, MiB/GiB for peak memory, tokens per request, or requests per minute. State the resource statistic: a bound on population p95 latency is not a bound on every request, and sampled RSS is not a proven maximum. Hard execution caps and stop rules complement planning bounds and their effect on quality must be included in the evaluated policy.

A finite evidence-ready candidate satisfying all constraints is admissible. A measured policy can fail an operational gate; an interval that cannot establish a quality gate leaves it unadmitted because evidence is inconclusive. Neither case proves that its unknown population quality is below the floor. Missing, invalid or incomparable evidence is reported as insufficient evidence, while unsupported capabilities are incompatible. The decision record preserves these reasons rather than collapsing them into one infeasible label.

Within the admissible set, minimize projected expected C_N(c). A conservative-objective variant instead minimizes C_N^U(c) and must be evaluated separately. It is literal worst-case optimization only if that bound is a supremum over a specified uncertainty set. In the primary variant, the upper allowance gates budget feasibility but does not replace expected cost in the objective. Least cost means least projected cost among evaluated admissible candidates, not the unknown globally smallest invoice.

For exact projected-cost ties, use lower C_N^U, then lower predeclared normalized resource utilization max_l(r_l^U/ b_l), then a lexicographically ordered immutable configuration identifier. Here max_l takes the largest dimensionless resource-to-limit ratio. Resource utilization is an optional tie-break; it cannot compensate for failed quality or determine the primary objective. Omit that tie-break when resource statistics are not meaningfully comparable. If monetary tolerances are used, define cost bins or a deterministic within-tolerance policy before selection and describe its result as near-minimum; pairwise epsilon comparisons can create non-transitive ties.

If no candidate is admissible, return no supported selection with a reason for each candidate. Do not lower q_{min} or enlarge delta after examining held-out outcomes. A reference can be used as a fallback only when it independently meets all mandatory gates and the fallback policy was frozen. Abstention, human escalation or request rejection otherwise requires its own evaluated policy. A held-out failure invalidates the operational claim and prompts a new protocol; it does not justify choosing a new winner on that same held-out set.

5.4 Evidence contract and software applicability

A prospective developer package separates task adapters; an immutable model/configuration registry; benchmark execution and quality evaluation; statistical bounds; cost and resource collectors; constraint validation; deterministic selection; decision logging; and reproducibility manifests. Each module consumes

versioned records rather than undocumented defaults. A decision receipt contains the task/population, case and split hashes, prompt and model/tokenizer revisions, reference, thresholds, outputs, native-unit metrics, paired bounds and error allocation, tariffs, projected and upper cost semantics, resource statistics, statuses, rejection reasons and tie-break trace. It links to raw records, source/environment hashes and deviations. Synthetic contract tests should verify missing/invalid evidence, ties, failed gates, empty sets and exact policy identity. Existing software implements the historical weighted selector; the new confidence-bound model/effort selector and its performance validation remain prospective.

5.5 Applying the equations

First, freeze the task, population, metric direction, configurations, reference, deployment horizon, thresholds, resource statistics and tie-breaks. Second, collect comparable predictions on the selection set and calculate observed quality with (1). Third, estimate the absolute-quality lower bound and paired degradation upper bound with the declared dependence and multiplicity treatment. Fourth, measure full-policy resource use and calculate projected cost with (2), together with separately justified cost and resource bounds. Fifth, check every constraint in (3), recording why each candidate is admitted, operationally rejected, incompatible or unsupported by evidence. Sixth, select the admitted candidate with the lowest projected cost. Freeze this choice and confirm it on untouched held-out cases; do not reselect on those cases after a failed confirmation.

6. Experimental Methodology

6.1 Completed supporting study

The completed Azure experiment evaluates anchored weighted pipeline selection, not Equation (3). Dry Bean supplies 13,611 rows, 16 features and seven labels; 68 repeated rows leave 13,543 exact descriptor groups. Grouped splits contain 6,804 training, 2,725 selection, 1,361 unused calibration and 2,721 held-out rows. Bike's original hourly member has 17,379 recorded rows and 165 absent calendar hours; chronological splits contain 6,442 training, 2,203 selection and 8,734 held-out observations. The task predicts the next recorded-clock hour with fixed parameters and previously observed count lags at 1, 24 and 168 hours. Missing hours are not zero rentals. Detailed methods and provenance are retained in Appendices B and C.

Logistic/ridge, 200-tree random forest and histogram boosting pipelines use training-fitted preprocessing and three declared hyperparameter settings per family. Tuning includes 60 fits; 16 deployment/baseline fits and 16 separate fresh refits are recorded. Development evidence is reused for tuning and selection, so its scores are potentially optimistic. All selectors share candidate families, action policies and point-estimate quality/resource gates. Four prespecified simulated operating profiles combine loss and latency utilities; their weights are study assumptions rather than elicited business costs.

Comparators are highest development quality, cheapest feasible warm-inference proxy, fixed linear and minimum development action loss. The latter prevents claiming a unique weighted-rule advantage when direct loss minimization makes the same choice. Three isolated development and three final profiling jobs use one thread and warm batches of 32 on Standard_D2s_v3 in East Asia. The recorded rate is the study-time USD 0.132/node-hour. The expense proxy excludes startup, features, IO, networking and support, and is not provider billing.

The frozen primary outcome is mean simulated action loss. Sixteen paired weighted-minus-comparator contrasts use 20,000 bootstrap draws and Bonferroni alpha 0.05/16. Bean resamples descriptor groups; Bike uses month-stratified circular calendar blocks of 168 hours. A separate 40,000-draw calculation checks endpoint stability. These conditional intervals rely on unverified dependence assumptions, do not include training/selection uncertainty and do not test the proposed quality non-inferiority constraints. Precision, practical-effect and exploratory sensitivity thresholds were frozen before held-out evaluation. All empirical computations were performed in Azure; the reported aggregate values are preserved.

6.2 Prospective validation of model and inference configurations

The following study is proposed and has not been run. Evaluate MBPP full-version Python tasks [17,18], not the differently sized sanitized subset. Revise the unused official 374 development/90 validation/500 test split before any generation: retain 74 official training problems for development, reserve the other 300 for selection, and combine them with the official 90 validation problems to obtain 390 selection problems. Keep all 500 official test problems untouched; IDs 1–10 remain optional fixed prompt examples. Assign whole duplicate/near-duplicate groups using outcome-independent rules, publish exact manifests and report any reduced counts. No selection task may have influenced prompt or harness tuning. These are local partitions, not evidence of independence from pretraining.

Compare exactly Qwen/Qwen3-4B and Qwen/Qwen3-8B in documented thinking and non-thinking modes [15,16]. Freeze model/tokenizer commit revisions and chat templates. Both cards state Apache-2.0; MBPP states CC BY 4.0. The fixed reference is 8B thinking, without assuming it is best or admissible. Proposed common hardware is one 24-GiB NVIDIA GPU, at least 64 GiB host RAM and 100 GiB storage; verify capacity in a development pilot. Use BF16 weights, concurrency one, an identical prompt and a 2,048 generated-token cap including reasoning. Thinking uses temperature 0.6/top-p 0.95; non-thinking uses 0.7/0.8; both use top-k 20 and min-p 0. These are complete-policy comparisons rather than an isolated causal test of thinking.

The primary outcome is one single-completion test-pass indicator per problem and configuration using the first registered seed; each problem has a separate derived random stream. Parsing failures, rejected execution, truncation and timeouts count as failures. Two additional registered generation seeds estimate stochastic sensitivity as secondary clustered analyses, not additional independent problems or pass@3. Freeze three base seeds (20261011–20261013), Python/runtime versions, stopping/parser rules and sandbox limits before selection. Execute code without network or secrets in a disposable container, capped at 1 GiB memory and 5 CPU seconds per test case. Generation timeout is 180 seconds; output beyond the cap is not repaired. Verify reference solutions and freeze defective-task exclusions before viewing model answers.

For a bounded research scenario, prespecify q_{min} = 0.60 and delta = 0.05 absolute proportion units. These are transparent study assumptions, not professional software acceptance standards; obtain stakeholder justification before any real-service claim. Allocate quality-family alpha = 0.05 across four absolute-floor and three non-identity reference-degradation claims. For primary binary outcomes, use one-sided exact binomial bounds [21]. Bound paired degradation conservatively by an upper bound on reference-pass/candidate-fail probability minus a lower bound on candidate-pass/reference-fail probability, allocating half the contrast error to each component. This preserves pairing, remains nondegenerate with zero disagreements and uses a union bound without assuming candidates are independent. It assumes independent, identically distributed problem-level trials under a declared benchmark-population model; related tasks must be quarantined/grouped or the inference narrowed. It is not a distribution-free guarantee for future software tasks.

A precision calculation illustrates the original design's limitation: with zero discordant primary outcomes and claim error 0.05/7, the conservative degradation upper bound is 0.0607 at 90 problems, versus 0.0143 at 390. These are analytical planning values, not measured model results. Under independent paired trials with discordance probability 0.10 and true degradation zero, exact multinomial planning gives only 23.5% power for this degradation gate at 390 selection problems and 62.5% at 500 held-out problems. Power also depends on unknown discordance; these assumptions are not model findings. The 390-problem design is a feasible improvement, not a power guarantee. Repeats do not fix a shortage of independent problems. If it cannot support admission, report no supported selection; never enlarge margins using observed outcomes.

Freeze the selector and baselines using selection evidence only: fixed reference, highest-quality operationally feasible configuration, cheapest operationally feasible configuration without the quality gates, and weighted utility with development-frozen anchors. Cheapest within the identical admissible set is an implementation

check, not an independent scientific competitor. A router or cascade is outside the minimum design and requires its own end-to-end study. On held-out data, allocate alpha = 0.05/2 to the selected configuration's absolute floor and paired degradation and apply the same exact procedure. Reuse predictions for identical policies and retain failed or inconclusive confirmation without reselection.

Measure every attempt's input/generated/reasoning tokens, end-to-end latency, failures, tools and retries. Separate cold loading from warm requests and repeat profiling in three isolated sessions with randomized configuration order. Proposed operational ceilings are request p95 <= 180 seconds, GPU memory <= 24 GiB, host memory <= 64 GiB and zero network/tool access during code execution. These are study limits, not derived customer needs. Resource allowance methods and enforced caps must be documented separately from quality confidence; missing justified evidence prevents admission. For the primary 1,000-request horizon, estimate full setup, allocated node-time, idle capacity, storage and transfer in USD. Reconcile experimental invoices separately. Tariff, available credit and expenditure cap must be verified and authorized before running; no paid work is launched here.

Sensitivity analysis uses development/selection evidence to vary the floor, degradation margin, operating budget and input-available length strata; freeze scenarios before held-out evaluation and treat multiple secondary claims as exploratory unless corrected. Publish prompts, split/model/environment hashes, test harness, raw traces, accounting, bounds and decision receipts with credentials removed. Public benchmark memorization, incomplete tests and family-specific coverage remain threats. Expanded tests or independently authored tasks provide separately reported sensitivity evidence, not proof that contamination is eliminated. The separate validation plan specifies the run manifest, affordability gate and stopping rules.

7. Results and Analysis: Completed Weighted-Selector Study

The following results are preserved from the completed cloud study. They do not apply Equation (3), and no new empirical model-and-effort result has been added.

7.1 Frozen decisions and feasibility

The weighted rule chose linear for both Bean profiles and histogram boosting for both Bike profiles. Its primary candidate did not change between the two profiles within either task. Bean's 5-ms resource gate admitted only the linear pipeline, forcing all five selectors to the same deployment. Bean quality admitted all learned families; validation quality and minimum validation loss chose boosting, whereas weighted, cheapest feasible and fixed default chose linear. Bike quality admitted all learned candidates; Bike resource excluded random forest. Weighted, validation quality and minimum validation loss chose boosting in both Bike profiles; cheapest feasible and fixed default chose linear. This design therefore shows no extra configuration-selection benefit over direct validation-loss minimization, and several contrasts are identities.

Q and R denote quality-priority and resource-constrained simulations. All settings and decisions were frozen before final evaluation. Tables 1 and 2 report task-specific predictive metrics; these metrics are not comparable across the two tasks.

Table 1. Dry Bean classification metrics for the frozen pipelines.

Primary deployment seed 20261011; 2721 held-out cases. All values below are copied from the cloud report and rounded only for presentation.

Candidate	Setting	Development macro_f1	Held-out macro_f1	Held-out accuracy
Linear	C=0.1	0.9346	0.9296	0.9151
Random forest	min_samples_leaf=2	0.9351	0.9290	0.9173
Histogram boosting	max_leaf_nodes=63	0.9394	0.9297	0.9162
Predictive baseline	Fixed baseline	0.0590	0.0591	0.2606

Table 2. Bike Sharing sequential one-hour-ahead prediction metrics; MAE and RMSE are rental-count units.

Primary deployment seed 20261011; 8734 held-out cases. All values below are copied from the cloud report and rounded only for presentation.

Candidate	Setting	Development mae	Held-out mae	Held-out rmse
Linear	alpha=10.0	36.3090	51.9836	78.2904
Random forest	min_samples_leaf=1	23.0501	52.6342	91.1781
Histogram boosting	max_leaf_nodes=31	21.1871	49.8233	84.3342
Predictive baseline	Fixed baseline	102.5070	168.1527	227.5747

7.2 Predictive evidence and operating loss

All learned candidates had better point predictive metrics than their simple baselines in these held-out observations; no additional baseline significance test is asserted. Bean learned macro-F1 values were very similar. For Bike Sharing, boosting had lower MAE than linear, while linear had lower RMSE. The later-period errors were larger than the development-selection errors. These differences alone do not identify a cause or guarantee future-period performance.

Table 3 records all selector outcomes, including repeated deployments. Loss is per bean or per recorded rental-count hour under the stated simulated costs. Selected pipelines passed their quality, median job-p95 latency and maximum sampled-memory confirmation checks on this workload. This is observed requirement confirmation, not a probabilistic service guarantee.

Table 3. Frozen selector outcomes and confirmation.

Profile	Selector	Pipeline	Held-out loss	Requirements met
Bean Q	Weighted	Linear	0.0849	Yes
Bean Q	Validation quality	Histogram boosting	0.0838	Yes
Bean Q	Cheapest feasible	Linear	0.0849	Yes
Bean Q	Fixed linear	Linear	0.0849	Yes
Bean Q	Minimum validation loss	Histogram boosting	0.0838	Yes
Bean R	Weighted	Linear	0.0849	Yes
Bean R	Validation quality	Linear	0.0849	Yes
Bean R	Cheapest feasible	Linear	0.0849	Yes
Bean R	Fixed linear	Linear	0.0849	Yes
Bean R	Minimum validation loss	Linear	0.0849	Yes
Bike Q	Weighted	Histogram boosting	83.6725	Yes
Bike Q	Validation quality	Histogram boosting	83.6725	Yes
Bike Q	Cheapest feasible	Linear	76.0994	Yes
Bike Q	Fixed linear	Linear	76.0994	Yes
Bike Q	Minimum validation loss	Histogram boosting	83.6725	Yes
Bike R	Weighted	Histogram boosting	65.7974	Yes
Bike R	Validation quality	Histogram boosting	65.7974	Yes
Bike R	Cheapest feasible	Linear	79.8514	Yes
Bike R	Fixed linear	Linear	79.8514	Yes
Bike R	Minimum validation loss	Histogram boosting	65.7974	Yes

7.3 Resource trade-offs

Table 4 retains development-selection and final-period resource aggregates. Latency is the median of three separate job-level p95 warm 32-case batch latencies; memory is the maximum sampled process RSS across the three jobs. The detailed cloud reports retain all individual jobs, request records and proxy estimates. These figures exclude network latency and raw feature construction. No estimator-only memory interpretation or measured economic saving is implied.

Table 4. Warm resource aggregates; expense is USD/1,000 cases as a partial inference proxy.

Task	Candidate	Dev p95 ms	Final p95 ms	Final RSS MiB	Expense proxy
Bean	Linear	2.489	2.183	192.51	2.200e-06
Bean	Random forest	21.048	21.091	216.21	2.090e-05
Bean	Histogram boosting	14.256	14.816	219.47	1.476e-05
Bean	Predictive baseline	2.576	2.555	217.14	2.305e-06
Bike	Linear	2.301	2.278	218.73	2.025e-06
Bike	Random forest	27.735	27.237	396.70	2.777e-05
Bike	Histogram boosting	4.463	4.602	217.64	3.874e-06
Bike	Predictive baseline	1.787	2.019	218.56	1.908e-06

7.4 Paired outcomes, multiplicity and precision

Table 5 and Figure 1 contain all 16 prespecified contrasts. Negative differences favor weighted selection. The 20,000-draw primary intervals use alpha 0.05/16 and condition on the fitted pipelines and stated group/calendar-block assumptions. Precision tests the declared maximum endpoint distance; Stability tests the separate 40,000-draw endpoint comparison. Exact zero intervals denote identical deployed pipelines, not statistical equivalence of different models.

For Bean quality, the weighted linear choice minus boosting had mean difference 0.0011 simulated units/bean and interval [−0.0099, 0.0129]. Its maximum endpoint distance was 0.0118, above the 0.01 target. Therefore no precise practical advantage or disadvantage was established. Every Bean resource contrast was an identical linear deployment.

For the Bike quality-priority profile, weighted boosting minus linear had a difference of 7.5731 weighted-count units/hour and adjusted conditional interval [3.0698, 12.1578]. The lower endpoint exceeded the illustrative adverse threshold of 2. The recorded maximum endpoint distance, 4.5847, and stability change, 0.1364, met the 5 and 0.5 targets. Under the declared conditional resampling assumptions, this is evidence of higher simulated loss for the weighted choice. Validation quality and minimum validation loss made that same boosting choice.

For the Bike resource-constrained profile, the boosting-minus-linear difference was −14.0540 with interval [−19.1464, −9.3371]. Its direction favored boosting, but maximum endpoint distance 5.0924 exceeded the 5-unit target. Stability passed. The prespecified precise-superiority claim is therefore withheld, despite the interval excluding zero. Reporting the unmet requirement avoids rounding away a decision boundary. No final outcome triggered reselection or changed primary hypotheses.

Table 5. Primary paired loss differences with adjusted conditional intervals.

Profile	Comparator	Difference	Interval	Precision	Stability
Bean Q	Validation quality	0.0011	[-0.0099, 0.0129]	No	Yes
Bean Q	Cheapest feasible	0.0000	[0.0000, 0.0000]	Yes	Yes
Bean Q	Fixed linear	0.0000	[0.0000, 0.0000]	Yes	Yes
Bean Q	Minimum validation loss	0.0011	[-0.0099, 0.0129]	No	Yes
Bean R	Validation quality	0.0000	[0.0000, 0.0000]	Yes	Yes
Bean R	Cheapest feasible	0.0000	[0.0000, 0.0000]	Yes	Yes
Bean R	Fixed linear	0.0000	[0.0000, 0.0000]	Yes	Yes
Bean R	Minimum validation loss	0.0000	[0.0000, 0.0000]	Yes	Yes
Bike Q	Validation quality	0.000	[0.000, 0.000]	Yes	Yes
Bike Q	Cheapest feasible	7.573	[3.070, 12.158]	Yes	Yes
Bike Q	Fixed linear	7.573	[3.070, 12.158]	Yes	Yes
Bike Q	Minimum validation loss	0.000	[0.000, 0.000]	Yes	Yes
Bike R	Validation quality	0.000	[0.000, 0.000]	Yes	Yes
Bike R	Cheapest feasible	-14.054	[-19.146, -9.337]	No	Yes
Bike R	Fixed linear	-14.054	[-19.146, -9.337]	No	Yes
Bike R	Minimum validation loss	0.000	[0.000, 0.000]	Yes	Yes

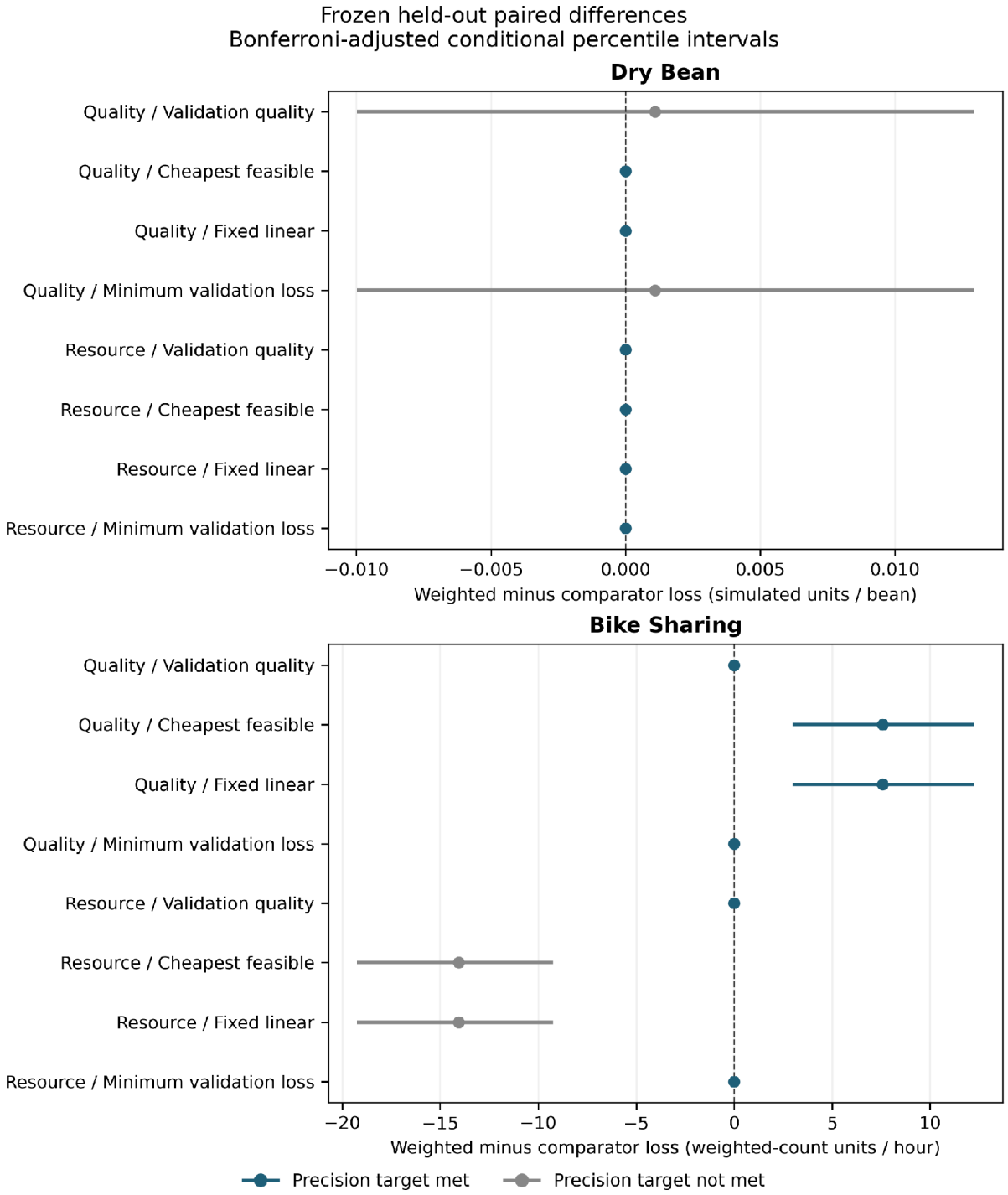


Figure 1. All 16 frozen weighted-minus-comparator mean-loss contrasts, with approximate conditional percentile intervals (20,000 resamples; Bonferroni alpha = 0.05/16). Blue marks attainment of the prespecified precision target, not significance or superiority; gray marks failure. Negative differences favor weighted selection. Zero-width intervals arise from identical deployments. Separate axes retain each task's simulated loss units. Intervals condition on frozen fitted models and the descriptor-group or month-stratified calendar-block sampling assumptions; they omit retraining and full reselection uncertainty.

7.5 Predeclared sensitivity and training variation

Bean weighted choices remained linear across all declared preference, unfavorable-anchor and latency-limit variants. The Bike weighted choice changed to linear at quality weight 0.2; weights 0.5 and 0.8 retained boosting. Halving Bike resource's 5-ms limit to 2.5 ms also selected linear. Unfavorable-anchor scale variants retained the primary weighted choices. These are predeclared exploratory choices using development inputs,

not test-selected replacements. Raw development Pareto sets were linear and boosting except Bean resource, whose feasible set contained only linear.

Table C3 reports temporal block diagnostics. They use nominal alpha 0.05 and 2,000 draws, unlike the adjusted 20,000-draw primary intervals; they cannot isolate block length while holding every inference setting constant. The 720-hour intervals can be especially narrow because long circular blocks nearly fix each month stratum's contribution. Their narrowness does not resolve future-year or cross-month uncertainty. Squared-utility and unclipped-score values remain exploratory diagnostics in the cloud report.

Table C2 reports five forest training seeds per task separately. Deterministic families retain one fit. These repetitions describe training randomness conditional on fixed partitions; they are not extra independent held-out observations and do not enlarge primary interval sample sizes.

7.6 Fresh cloud rerun and consistency audit

The separate fresh refit regenerated all 16 fitted configurations' held-out predictions and repeated the analysis. The final cloud audit passed a prediction agreement for classification actions/probabilities and regression actions/raw points at the frozen tolerances. Primary interval reproduction also passed. Exact split/group disjointness, held-out membership and pairing, source/artifact hashes, unchanged selectors and independent scalar-versus-vector loss arithmetic passed. These checks support implementation consistency on this cloud setup. They remain same-author automated checks and do not replace an independent scientific audit or peer review.

8. Discussion

The framework makes the practical goal explicit: quality is a prerequisite and complete projected monetary cost is the primary objective. This reduces the ambiguity of weighted scores whose preferences can trade quality against speed. It does not eliminate uncertainty, metric misspecification or distribution shift. A transparent selection receipt explains why a configuration qualifies; it cannot make invalid evidence reliable.

The supporting experiment warns against optimistic interpretation. Bean’s quality-profile linear-versus-boosting comparison was inconclusive, and its resource profile forced identical deployments. Bike boosting reduced MAE relative to linear yet had higher underprediction-heavy simulated loss. Under reversed costs its direction was favorable, but its declared precision target failed. Neither predictive quality nor a development loss ranking ensured lower held-out operational loss. The minimum-development-loss baseline also chose boosting, so the study provides no extra weighted-rule benefit over that direct comparator.

A confidence-bound cost optimizer changes the selection criterion, but no evidence here establishes that it would correct those empirical outcomes. In particular, the earlier loss intervals cannot be repurposed as macro-F1/MAE non-inferiority intervals, and its partial expense proxy cannot support complete financial savings. A retrospectively evaluated new policy on the same final data would be exploratory and would require fresh confirmation.

For software reuse, the most useful deliverable is the interface contract: record quality in its native units, reject unsupported settings, account for the complete policy, state uncertainty assumptions and preserve reasons for no selection. A modest package implementing this contract can be valuable without claiming general intelligence measurement or universal performance gains.

9. Threats to Validity and Limitations

Construct validity depends on task metrics and justified quality thresholds. Existing operating profiles use simulated error costs and hypothetical preferences. MBPP test passing is not full semantic correctness, security or professional software quality. Public code benchmarks are susceptible to training contamination, test incompleteness and memorization; neither a held-out local split nor additional tests alone removes that threat.

Internal validity in the completed study is limited by reuse of development data for tuning and selection, potential acquisition dependence beyond exact Bean duplicates, timezone-unspecified Bike timestamps and conditional uncertainty that omits training and full selection repetition. Month-stratified circular blocks assume dependence stability, fix month composition and omit cross-month/year variation. Narrow 720-hour exploratory intervals can result from conditioning artifacts. Multiplicity correction does not repair invalid bootstrap assumptions.

Operational validity is limited by warm batches, three profiling jobs without verified physical-host independence, sampled memory peaks and a partial CPU-dollar proxy. Successful aggregate confirmation is not a population service guarantee. Planning estimates are not invoices, and empty same-day billing responses do not establish zero expenditure. For the proposed study, shared-server allocations, price changes, token truncation, model revisions, quota failures and concurrency can alter admission and cost. Thinking-mode comparisons with different decoders evaluate complete policies and cannot isolate effort effects.

Statistical precision may be inadequate to establish non-inferiority for small margins. Finite candidate search and multiple task/stratum claims require explicit correction or independent confirmation. Zero differences from identical deployed policies are identities, not evidence of equivalence between different models. An interval crossing an admission boundary warrants an inconclusive status. Proposed resource/cost bounds require separate justification and runtime enforcement; they do not inherit coverage from quality intervals.

External validity is restricted to the measured populations and workloads. The completed two-task study does not validate code generation, LLM inference modes, adaptive routing, energy savings, economic savings or high-stakes deployment. Author-conducted automated audits support computational consistency but do not constitute independent peer review. A genuine deployment claim requires the new experiments, complete financial evidence, defensible uncertainty and an independently reviewable artifact.

10. Conclusion and Future Work

This paper specifies a unified task-conditioned selection framework with four components and one master constrained optimization problem. It chooses the least projected-cost configuration among evidence-ready candidates satisfying absolute quality, reference-relative non-inferiority, budget and resource requirements. The formulation applies established constrained decision principles and exposes unsupported or inconclusive decisions instead of silently weakening requirements.

The preserved Azure study documents same-author computational repeatability and adverse or inconclusive weighted-selection findings. It does not validate the new optimizer or demonstrate quality-preserving complete financial savings. The next step is the prespecified code-generation study with full cost measurement and untouched quality confirmation. Further work can evaluate independently justified difficulty routing, larger task populations and broader configurations after this bounded design is assessed.

Data, Code and Research Assistance

The original UCI datasets are available under CC BY 4.0 [9,10]. Source, frozen configuration, cloud aggregate reports and lineage are retained. Raw case predictions, fitted models and research inputs remain in a private Azure workspace; no public artifact repository or independent reproduction is asserted. An external researcher needs the versioned source, accessible inputs, split/prediction manifests, environment/container and execution instructions, plus authorized cloud access or a documented portable replacement. Aggregate checks alone cannot verify every case or reproduce training. The proposed protocol requires a new versioned implementation and artifact release. AI assistance supported implementation, manuscript preparation and technical editing; historical values originate from Azure reports. Pacharapol Pimpa remains responsible for verification and interpretation. This independent manuscript is not peer reviewed, and its university affiliation does not imply endorsement.

11. References

[1] Elkan, C. (2001). The Foundations of Cost-Sensitive Learning. Proceedings of IJCAI, 973–978. Author manuscript: https://cseweb.ucsd.edu/~elkan/rescale.pdf

[2] Hernández-Orallo, J., Flach, P., and Ferri, C. (2012). A Unified View of Performance Metrics: Translating Threshold Choice into Expected Classification Loss. Journal of Machine Learning Research, 13(91), 2813–2869. https://www.jmlr.org/papers/v13/hernandez-orallo12a.html

[3] Gardner, S., Golovidov, O., Griffin, J., Koch, P., Thompson, W., Wujek, B., and Xu, Y. (2019). Constrained Multi-Objective Optimization for Automated Machine Learning. DSAA 2019; author manuscript arXiv:1908.04909. https://arxiv.org/abs/1908.04909

[4] Chen, E., Dullerud, N., Niedermayr, T., Kidd, E., Senanayake, R., Koh, P. W., Koyejo, S., and Guestrin, C. (2026). Modeling Multi-Objective Tradeoffs with Monotonic Utility Functions. Proceedings of AISTATS, PMLR 300, 4753–4761. https://proceedings.mlr.press/v300/chen26i.html

[5] Dehghani, M., Tay, Y., Gritsenko, A. A., Zhao, Z., Houlsby, N., Diaz, F., Metzler, D., and Vinyals, O. (2021). The Benchmark Lottery. arXiv:2107.07002. https://arxiv.org/abs/2107.07002

[6] Cawley, G. C., and Talbot, N. L. C. (2010). On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. Journal of Machine Learning Research, 11, 2079–2107. https://www.jmlr.org/papers/v11/cawley10a.html

[7] Kapoor, S., and Narayanan, A. (2022). Leakage and the Reproducibility Crisis in ML-based Science. Author preprint arXiv:2207.07048. https://arxiv.org/abs/2207.07048

[8] Politis, D. N. (2003). The Impact of Bootstrap Methods on Time Series Analysis. Statistical Science, 18(2), 219–230. https://mathweb.ucsd.edu/~politis/StatSci03.pdf

[9] Dry Bean [Dataset]. (2020). UCI Machine Learning Repository. https://doi.org/10.24432/C50S4B

[10] Fanaee-T, H. (2013). Bike Sharing [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5W894

[11] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12(85), 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

[12] Chen, L., Zaharia, M., and Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176. https://arxiv.org/abs/2305.05176

[13] Ong, I., et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665. https://arxiv.org/abs/2406.18665

[14] Snell, C., Lee, J., Xu, K., and Kumar, A. (2024). Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. arXiv:2408.03314. https://arxiv.org/abs/2408.03314

[15] Yang, A., et al. (2025). Qwen3 Technical Report. arXiv:2505.09388. https://arxiv.org/abs/2505.09388

[16] Qwen Team. Qwen3-4B and Qwen3-8B official model cards. Model capabilities and licenses checked 11 October 2026. https://huggingface.co/Qwen/Qwen3-4B ; https://huggingface.co/Qwen/Qwen3-8B

[17] Austin, J., et al. (2021). Program Synthesis with Large Language Models. arXiv:2108.07732. https://arxiv.org/abs/2108.07732

[18] Google Research. Mostly Basic Python Problems: official split documentation and dataset card. Checked 11 October 2026. https://github.com/google-research/google-research/tree/master/mbpp ; https://huggingface.co/datasets/google-research-datasets/mbpp

[19] Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. arXiv:2305.01210. Evaluation implementation: https://github.com/evalplus/evalplus ; paper: https://arxiv.org/abs/2305.01210

[20] Schuirmann, D. J. (1987). A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. Journal of Pharmacokinetics and Biopharmaceutics, 15(6), 657–680. https://doi.org/10.1007/BF01068419

[21] Clopper, C. J., and Pearson, E. S. (1934). The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika, 26(4), 404–413. https://doi.org/10.1093/biomet/26.4.404

12. Appendices

Appendix A. Weighted utility retained for the completed study

These supporting equations describe the rule actually tested, not the new master optimization problem. With complete observed evidence and all original hard gates passed, map criterion x_j between fixed unfavorable anchor a_j and favorable anchor b_j:

$$u_j(m,t,s) \;=\; clip_{[0,1]}\left(\frac{x_j - a_j}{b_j - a_j}\right)$$

(A1)

Here m identifies a predictive pipeline, t its task, s its simulated operating profile and j a criterion. x_j is the measured criterion; a_j and b_j are distinct unfavorable and favorable anchors in matching units. For a lower-is-better criterion, b_j can be smaller than a_j. clip_[0,1] truncates its argument to [0,1], and u_j is a dimensionless utility. Clipping expresses indifference beyond the anchors; raw metrics and exploratory unclipped/squared utilities remain in the reports. w_j(t,s) are nonnegative, dimensionless preference weights summing to one:

$$S(m,t,s) \;=\; 100 \sum_j w_j(t,s)\, u_j(m,t,s),\; for\; F(m,t,s) \;=\; 1$$

(A2)

S(m,t,s) is a weighted preference score in 0–100 score points. F(m,t,s) is one when all original point-evidence gates pass and zero otherwise. The near-optimal set is:

$$A_\varepsilon \;=\; \{m :\; F(m,t,s) \;=\; 1\; and\; S(m,t,s) \geq S_{max} \;-\; \varepsilon\}$$

(A3)

S_{max} is the largest score among feasible models for that task/profile; epsilon is the allowed score-point shortfall, and A_ε is the corresponding feasible set. If no model passes the gates, S_{max} is undefined and the selector returns no selection. Apply (A1) to each criterion, combine weights with (A2), and use (A3) to collect the near-optimal candidates. The completed study uses epsilon = 0.5 score points and fixed linear, forest, boosting order within that set. This is near-optimal weighted selection, not exact least-cost selection. Every comparator uses the same original point-evidence feasibility gates; those gates do not instantiate the proposed simultaneous confidence constraints.

Appendix B. Detailed completed-study protocol

The following details document the completed weighted-selector experiment; its precision and feasibility gates differ from the proposed master framework.

B.1 Original-source audit and partitions

Dry Bean [9], originating in the Köklü and Özkan classification study identified by UCI, supplies 13,611 descriptor rows, 16 features and seven labels. The Azure audit found 68 rows repeating an exact descriptor vector beyond its first occurrence, leaving 13,543 groups. No invalid feature values, missing cells or conflicting labels within a group were found. Grouped stratification produced 6,804 training, 2,725 selection, 1,361 reserved calibration and 2,721 held-out rows. Entire descriptor groups stay in one partition. This addresses exact duplicates, while undocumented acquisition or batch dependence remains a limitation. The empirical task uses descriptors, not image acquisition or segmentation.

The original hourly Bike Sharing member [10] contains 17,379 rows, despite the repository page displaying 17,389 instances. Its completed calendar span has 165 absent hours. The audit found no duplicate timestamps, invalid values or missing recorded cells. The fixed split contains 6,442 training observations before October 2011, 2,203 selection observations in October–December 2011 and 8,734 reserved observations in 2012. Missing hours are not inserted as zero rentals.

Bike features contain known calendar information and count lags at 1, 24 and 168 hours on the dataset's recorded calendar clock. Target-hour weather, the current count components and the arbitrary record identifier are excluded. Calendar gaps create missing lag features handled by training-fitted median imputation. At evaluation hour h, only observations strictly before h are available. Source timestamps have no timezone or explicit daylight-saving disambiguation; the hour convention follows the supplied naive calendar timestamps rather than a verified UTC elapsed-time interpretation. The 2012 evaluation is sequential one-hour-ahead prediction with frozen parameters and actual earlier observed counts as they become available; it is not a forecast of the entire year made at its start. Recorded rentals do not reveal unmet demand when supply is constrained.

Archive, member, split and selection-case hashes were approved after the earlier cloud audit. Every stage checks those receipts. Raw archives necessarily contain all original labels in Azure, and original-data audit/split construction can read those labels. The stage flag final_test_accessed denotes consumption of held-out evaluation frames and targets, not cryptographic concealment of the original labels. Separation of final evaluation from tuning is enforced by the software gate and stage sequence, rather than distinct data-access accounts. Research inputs, labels, case predictions and model artifacts remain in Azure.

B.2 Bounded pipelines and tuning

The classification families are logistic regression, a 200-tree random forest and histogram gradient boosting. Regression uses ridge, a 200-tree random forest and histogram gradient boosting. Each pipeline includes an Azure execution guard, training-fitted median imputation with empty-feature retention, and training-fitted standard scaling. The simple classification baseline predicts training class priors with argmax; the regression baseline predicts the training mean. All classification actions use the same uncalibrated argmax rule. All regression actions clip mean-oriented point predictions to zero. Neither is claimed to minimize arbitrary asymmetric decision loss.

Each learned family receives three declared settings: regularization 0.1/1/10 for linear models, forest minimum leaf size 1/2/4, and boosting maximum leaves 15/31/63. Remaining pilot settings are fixed, including 100 boosting iterations without early stopping. Bean tuning uses the grouped selection partition and macro-F1. Bike tuning minimizes MAE across expanding validation windows: May–June, July–September and October–December 2011, with training restricted to earlier observations. Folds have equal weight. Random forests use seeds 20261011–20261013 in tuning and five seeds 20261011–20261015 for frozen-configuration variability. The primary deployment seed is 20261011; deterministic families are not redundantly repeated.

This design contains 60 tuning fits and 16 deployment/baseline fits. Another 16 fresh fits are reserved for reproducibility, distinct from synthetic software tests and the earlier eight-candidate pilot. Deployment fits use only the original training partitions; selection and reserved calibration observations are not added to training. Tuning and selection reuse development evidence, so reported development quality can be optimistic. Final outcomes provide the separate evaluation.

B.3 Simulated operating profiles and comparator fairness

Each task has a quality-priority profile and a resource-constrained profile. Bean quality uses unit misclassification cost; its resource profile triples the cost of an error whose true label is BOMBAY. Bike quality weights underprediction twice overprediction; its resource profile reverses that ratio. All costs,

preferences, anchors, resource requirements and precision targets are illustrative assumptions, without stakeholder elicitation, a real service contract or monetary interpretation.

Quality-priority scores weight loss/latency at 0.8/0.2; resource profiles use 0.3/0.7. Bean loss anchors are 0.02/0.30 and Bike anchors 15/150, favorable/unfavorable respectively. Latency anchors are 0.5/50 ms per warm batch of 32 cases. Hard limits are batch-p95 latency ≤50 ms for quality profiles or ≤5 ms for resource profiles, sampled process memory ≤512 MiB, and macro-F1 ≥0.85 for Bean or MAE ≤60 rentals/hour for Bike. Score tolerance is 0.5 points. Comparator tolerances are macro-F1 0.001 or MAE 0.5, estimated expense 10^{-8} USD/1,000 cases, and validation loss 0.001 for Bean or 0.5 for Bike.

The weighted selector is compared with highest validation quality, cheapest feasible inference proxy, fixed linear default, and minimum validation action loss. The last comparator is necessary because loss is also a primary score input: comparison only with a quality metric could exaggerate the value of decision-aware scoring. Every selector uses the same learned candidates, evidence, action policies and feasibility gates. A missing or infeasible fixed default remains undefined, rather than receiving a substituted model.

B.4 Separate resource measurement

Three separate development profiling jobs run without concurrent training. Every candidate is loaded in a fresh child process, with one model thread, two warm-up blocks, five measurement blocks and batch size 32. The setup is a dedicated Standard_D2s_v3 Azure worker in East Asia with a maximum of one node. The selection latency is the median of three job-level full-batch p95 measurements; memory is the maximum sampled process RSS across those jobs. Individual job measurements are retained. RSS sampling can miss a short-lived allocation peak and includes the Python process rather than estimator memory alone.

Expense is an estimated warm inference CPU-dollar proxy: measured request seconds per 1,000 cases multiplied by the verified rate of USD 0.132/node-hour. It excludes startup, feature construction, IO, network and supporting services, and is not actual Azure expenditure. Later three-job profiling on final-period inputs confirms the frozen requirements. These jobs measure process and job variation on this setup; they do not verify different physical hosts, network request latency or population-wide deployment performance.

B.5 Freeze, paired uncertainty and predeclared sensitivity

A cloud-generated freeze binds exact source/configuration/data hashes, fitted configurations, primary deployment seed, resource evidence, comparator decisions and development precision diagnostics. Its exact file hash is reviewed before held-out access. This is an internally prespecified and timestamped protocol, not an externally registered preregistration. No setting, primary selector or requirement is changed using final outcomes.

The primary outcome is simulated action loss on the held-out cases. Bean loss is the true-class weight times the indicator of a wrong action. Bike loss is c_{under} max(y−a,0) + c_{over} max(a−y,0), where y is the observed rental count, a the nonnegative prediction, and c_{under} and c_{over} the declared underprediction and overprediction weights. max denotes the larger argument; weights are dimensionless and loss retains count units. These are simulated penalties rather than billed expenditure. Sixteen primary contrasts compare weighted selection with four comparators across four profiles, on identical case identifiers and targets. The difference is weighted loss minus comparator loss; negative values favor the weighted rule.

Bean uncertainty resamples exact descriptor groups and uses row-weighted group loss sums. Bike uncertainty uses month-stratified circular blocks on a complete recorded calendar-hour grid, with a predeclared weekly block of 168 hours. Missing hours contribute neither observations nor loss. Calendar cycles motivate the weekly block; development loss autocorrelations at 1/24/168/336 hours are recorded before the freeze. Conditional inference assumes approximately stable dependence within each month, which is not guaranteed by this observational series. Month stratification fixes the observed period's month composition, does not

reconstruct dependence across month boundaries, and cannot quantify uncertainty over another year's structural change.

Primary percentile intervals use 20,000 resamples and Bonferroni alpha 0.05/16. A separate seed with 40,000 resamples checks endpoint stability. Nominal familywise coverage remains approximate and conditional on the underlying bootstrap assumptions. Identical deployed models have exactly zero paired differences; that identity is not an equivalence claim for different models. Meaningful differences are 0.01 Bean simulated units and 2 Bike weighted-count units/hour. Desired maximum endpoint distances from the estimate are 0.01 and 5 respectively; endpoint stability tolerances are 0.002 and 0.5. These thresholds are hypothetical precision requirements, not validated business requirements. Unmet targets narrow claims rather than trigger test-driven retuning or sample inflation.

Exploratory diagnostics vary quality weights over 0.2/0.5/0.8, both unfavorable anchors over factors 0.75/1.25, and the latency limit over factors 0.5/2. They also report squared utilities, unclipped scores and raw development Pareto sets. Alternative choices use development inputs exclusively, and their final losses are diagnostic, not new primary winners. Bike block-length sensitivity uses 24/336/720 hours and 2,000 draws with nominal 95% intervals, distinguished from multiplicity-adjusted primary inference. A block approaching a full month can make that stratum's contribution nearly fixed under circular resampling; an unusually narrow long-block sensitivity interval is a conditioning artifact rather than stronger coverage evidence. Training-seed results are reported separately from case/time uncertainty. The intervals condition on the frozen fitted pipelines; they do not resample training data or repeat the full model-selection procedure.

B.6 Cloud execution and reproducibility checks

All dataset acquisition, processing, fits, predictions, profiling, software tests, uncertainty calculations and consistency analyses occur in Azure. The Mac prepares source and documents, submits bounded jobs and reads cloud-generated aggregate reports. Azure software checks passed 119 cases with no failures or skipped tests; two warnings were recorded: an intentional malformed-data fixture warning and a pandas future-compatibility notice for descriptor-group factorization. The pinned runtime completed successfully; compatibility with a future pandas version is not asserted.

The versioned environment uses Python 3.11.11, NumPy 2.2.6, SciPy 1.15.3, pandas 2.2.3, scikit-learn 1.6.1 [11], joblib 1.4.2, threadpoolctl 3.6.0, psutil 7.0.0, pytest 8.3.5 and matplotlib 3.10.6. The Microsoft Linux base image is pinned by digest. Dependencies, sources, configurations and stage records are retained. A separate cloud job freshly refits all frozen configurations, regenerates predictions and reruns the analysis. Prediction agreement is checked at absolute/relative tolerance 10^{-8}; interval agreement uses absolute 10^{-6} and relative 10^{-8}. A later cloud audit checks split/group separation, held-out membership and paired ordering, unchanged selectors, artifact hashes and a separate scalar implementation of loss arithmetic.

These same-author automated checks support computational consistency; they do not replace external scientific review. Planning costs are distinct from actual provider billing. No complete billed-expenditure savings result is reported.

Appendix C. Traceability and extended completed results

Exact pretest freeze SHA-256: 73218eb8907982a5935949af0de6daed472f1a9a312fe639c0c65ee547ff64b7.

Study configuration SHA-256: 8790f2ac63eb52899f13f21b58d90e06a33a7e22cf8a25c7630704d40d115675.

Base image digest: sha256:15e3cf1ec54d3051024b0e4d12fca577b343e0a62d89167e40ffba2dcb4eb414. Environment: task-conditioned-pilot version 2. Resolved Python dependencies and all 19 research-source hashes are retained with the cloud receipts.

Table C1 records cloud stage lineage; job names are artifact identifiers, not scholarly citations. The original analysis predates the fresh rerun. The later same-author consistency report supplies the reported computational agreement checks, not external reproduction.

Table C1. Verified cloud stage lineage (reproducibility identifiers, not scholarly references).

Stage	Azure job
study_smoke	nifty_longan_8zj45xmpq7
development	strong_leaf_cs1jr8jn7w
profile_dev_1	keen_heart_y25580kwyk
profile_dev_2	ivory_parang_f9k58z3603
profile_dev_3	eager_kite_dnhl28mkkx
freeze	quirky_crayon_my5lsvt43b
final	funny_door_qyrvlqnyff
profile_final_1	mighty_tangelo_0ytpvcnj9y
profile_final_2	cool_giraffe_xtgh3srlv2
profile_final_3	icy_insect_t6p570n3f7
analysis	heroic_match_bmkjf9vn6m
rerun	salmon_loquat_88zq5jwy2v
consistency	blue_boat_ktckb31n2t

Table C2. Individual training-seed held-out metrics; no local averages or uncertainty estimates are added.

Task	Candidate	Seed	Metric	Reported value
Bean	Histogram boosting	20261011	macro_f1	0.929686
Bean	Linear	20261011	macro_f1	0.929620
Bean	Predictive baseline	20261011	macro_f1	0.059059
Bean	Random forest	20261011	macro_f1	0.928962
Bean	Random forest	20261012	macro_f1	0.928531
Bean	Random forest	20261013	macro_f1	0.927408
Bean	Random forest	20261014	macro_f1	0.928258
Bean	Random forest	20261015	macro_f1	0.927125
Bike	Histogram boosting	20261011	mae	49.823301
Bike	Linear	20261011	mae	51.983584
Bike	Predictive baseline	20261011	mae	168.152684
Bike	Random forest	20261011	mae	52.634243
Bike	Random forest	20261012	mae	53.527127
Bike	Random forest	20261013	mae	53.093890
Bike	Random forest	20261014	mae	52.341917
Bike	Random forest	20261015	mae	52.879810

Table C3. Bike versus linear calendar-block diagnostics, with different nominal alpha/draw counts shown explicitly.

Profile	Block hours	Draws	Alpha	Conditional interval
Bike Q	168 primary	20000	0.003125	[3.0698, 12.1578]
Bike Q	24	2000	0.05	[4.3964, 10.7574]
Bike Q	336	2000	0.05	[4.6655, 10.3282]
Bike Q	720	2000	0.05	[6.9957, 8.1388]
Bike R	168 primary	20000	0.003125	[-19.1464, -9.3371]
Bike R	24	2000	0.05	[-17.3910, -11.0054]
Bike R	336	2000	0.05	[-16.8982, -11.3341]
Bike R	720	2000	0.05	[-14.6332, -13.4477]

Detailed source/data hashes, three-job resource records, development precision/autocorrelations, all sensitivity variants, raw Pareto sets and squared/unclipped diagnostics are retained in the unchanged cloud aggregate reports. The supplemental presentation figure reads only those aggregate values; its source and input-result hashes are retained separately.